OFFSIDE LABS

Stake for Fee

Release 0.1.2

Smart Contract Security Assessment

November 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie



Contents

1 About Offside Labs **2**

2 Executive Summary **3**

3 Summary of Findings **5**

4 Key Findings and Recommendations **6**

4.1 Locked Fees Become Unclaimable After the Final Exit 6

4.2 Frequent claim_fee_crank Prevents Full Fee Unlocking 7

4.3 Informational and Undetermined Issues 8

5 Disclaimer **10**

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *Stake For Fee* smart contracts, starting on October 20th, 2026, and concluding on November 3rd, 2026.

Project Overview

Stake for Fee Release 0.1.2 deprecates vault creation, staking, and unstake requests and cancellations. Users can withdraw their entire stake and receive pending rewards in both tokens directly, without restaking or a withdrawal delay. Existing unstake requests become immediately redeemable, and users with no remaining stake can claim pending rewards. Fee estimation adopts internal liquidity token supply accounting from Dynamic AMM and supported Dynamic Vault versions. Permissionless fee distribution and administrative controls remain available. The update also adds support for closing empty staking escrows, zero balance unstaking requests, and fully exited vault state accounts with their associated lists, returning account rent to users and vault creators.

Audit Scope

The assessment scope contains mainly the smart contracts of the stake_for_fee program for the *Stake For Fee* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- *Stake For Fee*
  - Codebase: `https://github.com/MeteoraAg/stake-for-fee`
  - Release 0.1.2: `PR-41`
  - Commit Hash: 21adfa42f5e437dfcd86f00eedce94f314d2552b

The issues described in this report have been resolved by the following commit:

- *Stake For Fee*
  - Codebase: `https://github.com/MeteoraAg/stake-for-fee`
  - Commit Hash: 17b481b4c52bd83fb65bf6f288c8a7b3f00b0363

We listed the files we have audited below:

- programs/stake-for-fee/src/constant.rs
- programs/stake-for-fee/src/errors.rs
- programs/stake-for-fee/src/events.rs
- programs/stake-for-fee/src/extensions/dynamic_amm_ext.rs
- programs/stake-for-fee/src/extensions/dynamic_vault.rs
- programs/stake-for-fee/src/instructions/cancel_unstake.rs
- programs/stake-for-fee/src/instructions/claim_fee.rs
- programs/stake-for-fee/src/instructions/claim_fee_and_withdraw_full.rs

- programs/stake-for-fee/src/instructions/claim_fee_crank.rs
- programs/stake-for-fee/src/instructions/initialize_stake_escrow.rs
- programs/stake-for-fee/src/instructions/initialize_vault.rs
- programs/stake-for-fee/src/instructions/mod.rs
- programs/stake-for-fee/src/instructions/stake.rs
- programs/stake-for-fee/src/instructions/unstake.rs
- programs/stake-for-fee/src/instructions/update_seconds_to_full_unlock.rs
- programs/stake-for-fee/src/instructions/withdraw.rs
- programs/stake-for-fee/src/lib.rs
- programs/stake-for-fee/src/states/stake_escrow.rs
- programs/stake-for-fee/src/states/top_list_metadata.rs
- programs/stake-for-fee/src/utils.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 1 medium issues
- 1 low issues
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Locked Fees Become Unclaimable After the Final Exit	Medium	Fixed
02	Frequent claim_fee_crank Prevents Full Fee Unlocking	Low	Acknowledged
03	Outdated Version Labels in Bundled IDLs	Informational	Fixed
04	Existing Lock Escrow Fees May Remain Unclaimable	Informational	Acknowledged

4 Key Findings and Recommendations

4.1 Locked Fees Become Unclaimable After the Final Exit

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

`claim_fee_and_withdraw_full` IX applies the normal gradual fee release, claims the user's pending rewards, and removes their entire stake. It does not fully distribute the remaining `locked_fee_a` and `locked_fee_b` before the last staker exits.

After the last staker exits, `effective_stake_amount` becomes zero. Subsequent fee distribution calls return without releasing the remaining locked fees:

```
272 let effective_total_stake_amount = self.effective_stake_amount;
273 if effective_total_stake_amount == 0 {
274     // if total stake is zero, or haven't reach fee distribution time, we
        ↪  will preserve the fee for future stakers
275     self.update_last_updated_at(current_time);
276     return Ok((0, 0));
277 }
```

programs/stake-for-fee/src/states/vault.rs#L272-L277

This behavior preserves fees for future stakers. However, staking and restaking are now disabled as part of the deprecation, so no future stake can restore fee distribution.

Impact

Undistributed rewards can become inaccessible through the current program after all stakers exit. The affected funds are the global locked fees remaining in the vault. Rewards already credited to individual users remain claimable.

Recommendation

It is recommended to remove the fee unlock delay during deprecation and immediately distribute existing locked fees and newly collected fees to eligible stakers. Settle each user's rewards before reducing their stake, applying the same accounting to both cranking and withdrawal paths.

Alternatively, introduce an authorized recovery IX that transfers the remaining undistributed locked fees to a designated recipient after all stakes have been withdrawn.

Mitigation Review Log

Fixed in the commit 5464c15d317eac6097615ed05cfb71bdfb7cafe3.

4.2 Frequent claim_fee_crank Prevents Full Fee Unlocking

Severity: Low	Status: Acknowledged
Target: Smart Contract	Category: Design

Description

The permissionless `claim_fee_crank` IX releases fees using the remaining locked balance and the time elapsed since the previous distribution update:

```
225 let fee_a_amount_released = u128::from(self.locked_fee_a)
226     .checked_mul(seconds_elapsed.into())
227     .ok_or_else(math_error!())?
228     .checked_div(seconds_to_full_unlock.into())
229     .ok_or_else(math_error!())?
230     .try_into()
231     .map_err(type_cast_error!())?;
```

programs/stake-for-fee/src/states/vault.rs#L225-L231

After each distribution, the program resets `last_updated_at`, even when the released amounts are zero:

```
310 self.update_last_updated_at(current_time);
```

programs/stake-for-fee/src/states/vault.rs#L310-L311

Consequently, subsequent calls apply the same unlock duration to a progressively smaller balance.

For example, assume active stakers, 100 locked tokens, a six hour unlock period, and no additional fees:

Only 75 tokens are released after six hours. Without the intermediate call, all 100 tokens would be released at that time.

As the balance decreases, rounding can eventually produce zero whenever `remaining_locked_fee × seconds_elapsed < seconds_to_full_unlock`. Each call still resets the timestamp, preventing sufficient elapsed time from accumulating while frequent cranking continues.

Impact

Any caller can delay reward distribution through frequent cranking and keep small residual balances locked while those updates continue. Releasing the entire remainder requires an uninterrupted unlock period between distribution updates.

Recommendation

It is recommended to calculate cumulative unlocked fees from a fixed schedule and subtract the amount already released. Preserve fractional progress so that repeated calls, including calls releasing zero fees, do not delay unlocking.

Mitigation Review Log

Meteora Team: ACK. It's design choice.

4.3 Informational and Undetermined Issues

Outdated Version Labels in Bundled IDLs

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

The bundled DAMM and Dynamic Vault IDLs declare versions `0.5.2` and `0.9.1`, respectively, while incorporating account fields introduced in later releases. Other definitions remain outdated or incomplete, including the DAMM `withdraw_protocol_fees` signature and the missing Dynamic Vault migration instruction. Although the account layouts and fee claim interface used by stake-for-fee match the reviewed upstream versions, the complete IDLs do not represent those versions consistently.

It is recommended to align the IDL contents and metadata with pinned upstream releases and document any retained interface subsets or local modifications.

Mitigation Review Log

Fixed in the commit 34041d9a6990e061a21e81680964d6e3cca98502.

Existing Lock Escrow Fees May Remain Unclaimable

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Design

DAMM allows a `LockEscrow` to be created with the future `FeeVault` PDA as its owner before that FeeVault exists. If escrow creation and LP locking succeed in a separate TX, the upgrade prevents the remaining initialization step from completing because `initialize_vault` IX now unconditionally returns `VaultError::Deprecated`.

The existing DAMM IXs cannot claim fees from such an escrow without an initialized FeeVault.

It is recommended to update the documentation to instruct users and partners to stop creating stake-for-fee vaults and their associated DAMM LockEscrows.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

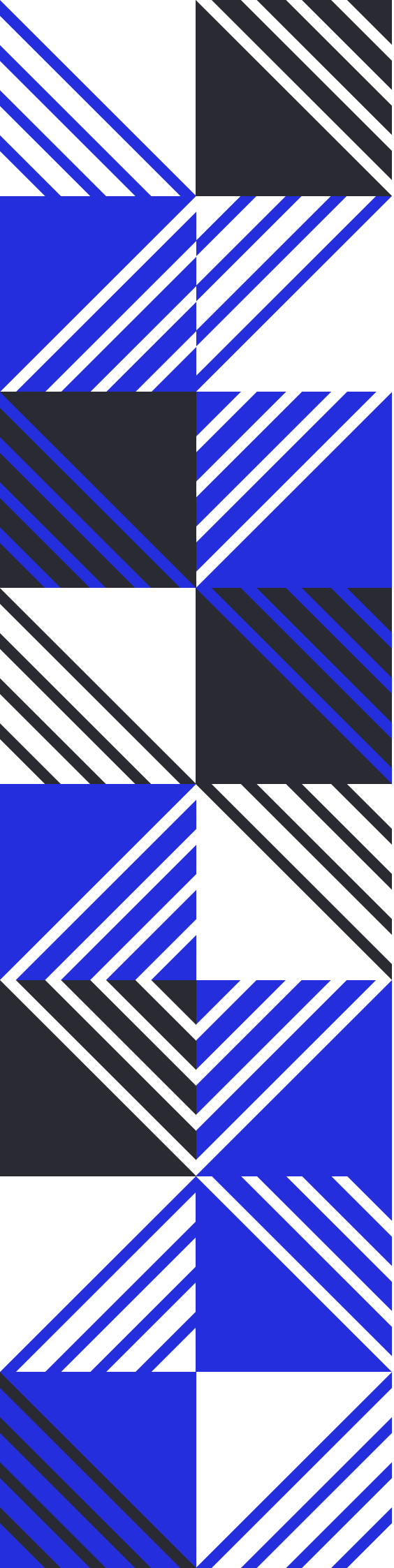
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs